

# lota-dsl

```
#include "../include/dsl.h"
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```
// Função que define se uma linha deve ser lida ou ignorada
int linha_valida(char *linha) {
    return (linha[0] != '#' && linha[0] != '\n' && linha[0] != '\0' &&
            strlen(linha) > 1);
}
```

*Se não começa com # e tamanho superior a 1 retorna verdadeiro*

```
// Função que prepara uma nova pilha de cartas no tabuleiro quando
encontrar o comando INIT
void tratar_init(EstadoJogo *e, char *tipo, int n) {
    e->pilhas = realloc(e->pilhas, (e->total_pilhas + 1) * sizeof(Pilha));
    Pilha *p = &e->pilhas[e->total_pilhas];
    strcpy(p->nome_tipo, tipo);
    p->quantidade = n;
    if (n > 0) {
        p->cartas = malloc(n * sizeof(Carta));
    } else {
        p->cartas = NULL;
    }
    e->total_pilhas++;
}
```

*Estica o espaço para caber mais uma pilha*  
*Aloca para a pilha alocada de criar*  
*Copia o tipo e número de cartas para a pilha*  
*Apenas se a pilha tiver cartas é que aloca memória para elas*  
*Atualizamos de criar uma pilha por isso soma +1*

```
// Função que regista as regras de como as cartas se podem mover entre
pilhas quando encontra o comando MOV
void tratar_mov(EstadoJogo *e, char *orig, char *dest, char *flags) {
    int i = 0;
    while (flags[i] != '\0') {
        if (flags[i] >= 'a' && flags[i] <= 'z') flags[i] -= 32;
        i++;
    }
    e->regras = realloc(e->regras, (e->total_regras + 1) * sizeof(Regra));
    strcpy(e->regras[e->total_regras].origem, orig);
    strcpy(e->regras[e->total_regras].destino, dest);
    strcpy(e->regras[e->total_regras].flags, flags);
    e->total_regras++;
}
```

*Aloca memória para +1 regra*  
*Fazemos a conversão de letras para maiúsculas*  
*Copia a origem, o destino e as flags para a memória*  
*Atualiza o nº de regras*

```
// Função auxiliar para guardar as regras automáticas
void tratar_auto(EstadoJogo *e, char *orig, char *dest, char *flags) {
    int i = 0;
    while (flags[i] != '\0') {
        if (flags[i] >= 'a' && flags[i] <= 'z') flags[i] -= 32;
        i++;
    }
    e->auto_regras = realloc(e->auto_regras, (e->total_auto_regras + 1) *
        sizeof(Regra));
    strcpy(e->auto_regras[e->total_auto_regras].origem, orig);
    strcpy(e->auto_regras[e->total_auto_regras].destino, dest);
    strcpy(e->auto_regras[e->total_auto_regras].flags, flags);
    e->total_auto_regras++;
}
```

*Função exatamente igual à anterior mas que guarda em outra variável*

fizemos isto para o automatico ter as próprias regras e não acontecer mais o bug que tivemos de fazer movimentos automáticos que não deviam acontecer



// Função que percebe qual é o comando específico dentro de uma linha válida

```
void processar_comando(char *linha, EstadoJogo *e) {  
    char p1[50], p2[50], p3[50];  
    int n;  
    if (sscanf(linha, "TIPO %s %s", p1, p2) == 2) { ← tipo não faz nada  
    }  
    else if (sscanf(linha, "INIT %s %d", p1, &n) == 2) { } → Se começar por INIT  
        tratar_init(e, p1, n); → manda para a função  
        que trata dele  
    }  
    else if (sscanf(linha, "MOV %s %s %s", p1, p2, p3) == 3) { } → Se começar por MOV  
        tratar_mov(e, p1, p2, p3); → manda para a função  
        que trata dele  
    }  
    else if (sscanf(linha, "AUTO %s %s %s", p1, p2, p3) == 3) { } → Se começar por AUTO  
        tratar_auto(e, p1, p2, p3); → manda para a função  
        que trata dele  
    }  
    else if (sscanf(linha, "BARALHOS %d", &n) == 1) { } → guarda o nº de baralhos  
        e->num_baralhos = n;  
    }  
    else if (sscanf(linha, "JOGO %s", p1) == 1) { } → Diz que jogo está a carregar  
        printf("Carregando: %s\n", p1);  
    }  
}
```

// Função que carrega o jogo escolhido

```
int carregar_paciencia(const char *nome_ficheiro, EstadoJogo *e) {  
    FILE *f = fopen(nome_ficheiro, "r"); ← Pede ao Computador para abrir o ficheiro  
    char linha[256]; ← Reserva um espaço para ler 256 caracteres  
    int sucesso = 0;  
    if (f != NULL) { ← Se conseguir abrir  
        while (fgets(linha, sizeof(linha), f)) { ← Com fgets ele lê linha a linha  
            if (linha_valida(linha)) processar_comando(linha, e); ← Se a linha for  
            } → válida, processa o  
            comando  
        }  
        fclose(f); ← fecha o ficheiro  
        sucesso = 1; ← muda o sucesso  
    }  
    return sucesso;  
}
```